

zkPoP: a machine-checked core for a self-hosting zero-knowledge skill, with N -of- M image carriers

Generated from the canonical manifest

Protocol **zkPoP/1** revision **5e231eebf317**

Abstract

This report accompanies a Lean 4 development that machine-checks the mathematical core of a “self-hosting zero-knowledge skill”: a system that publishes the *statements* of a formal library together with a zero-knowledge argument of knowledge of the withheld proof material, and distributes the resulting package over M image carriers of which any N suffice to reconstruct it. Every claim of the design that is mathematical – soundness of the checker, faithfulness of its arithmetisation, hiding and binding of the commitment, completeness, special soundness and perfect honest-verifier zero knowledge of the argument, N -of- M recovery and sub-threshold privacy, and the preservation of knowledge soundness under recursive self-verification – is proved in Lean with no `sorry` and with axiom closure limited to Lean’s three standard axioms. The report also states, and proves, the limitation of the design: recursion transports soundness rather than creating it, so semantic content must come from a theorem about the base relation.

1 What is claimed

The artifact supports exactly the following composition.

1. **Lean semantic layer.** A type checker for a typed object language is proved sound: acceptance implies the term evaluates to a value of the claimed type.
2. **Arithmetisation.** The checker is compiled into a constraint system over a field, and the compilation is proved *faithful*: the circuit is satisfiable exactly on the well-typed declarations. Hence a satisfying assignment (the private execution trace) entails the semantic claim.
3. **Argument layer.** A Pedersen commitment to the digest of the withheld material, plus a Fiat–Shamir Schnorr argument of knowledge of its opening bound to the statement root, the environment hash and the checker identifier. Completeness, special soundness (an explicit extractor) and perfect honest-verifier zero knowledge are proved.
4. **Recursion.** An abstract model of argument systems in which the outer proof verifies the inner one; knowledge soundness and completeness are preserved at every depth, and a single aggregated argument covers a whole library.
5. **Distribution.** The package is erasure-coded and rendered as M PNG carriers; any N reconstruct it, and any $N - 1$ are consistent with every possible payload.

What is *not* claimed: that the cryptographic layer alone establishes mathematics. Theorem `ZkPoP.zk_acceptance_alone_insufficient` makes this precise by exhibiting a perfectly knowledge-sound, self-verifying system whose acceptance implies nothing at all.

2 The published artifact

Declarations published (statements in clear)	65
Statement Merkle root	6d35aa87712f5a4fbbea2ca37c1b0e66...
Environment hash	94277ddd30bcc6f2a3e638625a08d559...
Checker image identifier	f20db6867b5c2946a576ecac9284d48c...
Commitment group	2048-bit MODP safe prime, order- q QR subgroup
Commitment $C = g^m h^r$	2637027589897733331431809504027579218619...
Challenge c (Fiat–Shamir)	8620406916060906851128375186709528420302...
Responses (z, w)	152551919273987079378307..., 121097718007662073577070...
Package size (compressed)	9246 bytes
Carriers	7 PNG images, threshold 4

The verification equation is $g^z h^w = t \cdot C^c \pmod{p}$ with $c = H(\text{context} \parallel C \parallel t \parallel \text{statement root})$. The witness – the proof material – never leaves the prover: it enters only as the exponent m of a perfectly hiding commitment.

3 Machine-checked results

All statements below are proved in the accompanying Lean development.

Declaration	Module	Statement
<code>ZkPoP.Tm.checker_sound</code>	Lang	If the type checker accepts a term at type τ then evaluation succeeds and returns a value of type τ (the semantic bridge).
<code>ZkPoP.typeSat_iff</code>	Arith	The arithmetic constraint system for type checking is satisfiable with output tag t <i>iff</i> the checker accepts the term at the type with tag t (faithful arithmetisation).
<code>ZkPoP.typeSat_sound</code>	Arith	A satisfying assignment of the circuit entails the semantic claim.
<code>ZkPoP.rootR1CS_sat_iff</code>	Arith	An explicit R1CS instance is satisfiable exactly when the public quadratic has a root; the root lives only in the witness.
<code>ZkPoP.horner_root_iff_trace</code>	Arith	Arbitrary degree: the Horner trace constraints are satisfiable with output wire 0 <i>iff</i> the witness is a root of the public polynomial.
<code>ZkPoP.pedersen_perfectly_hiding</code>	Pedersen	For any two messages there is a bijection of the randomness carrying the commitments of one onto those of the other: the commitment is independent of the message.
<code>ZkPoP.pedersen_binding_extract_dlog</code>	Pedersen	Any two distinct openings of one commitment yield the discrete logarithm of h in base g ; binding therefore reduces to discrete log.
<code>ZkPoP.Schnorr.completeness</code>	Schnorr	Honest transcripts always verify.
<code>ZkPoP.Schnorr.special_soundness</code>	Schnorr	Two accepting transcripts with a common announcement and distinct challenges yield, via an explicit extractor, a genuine opening: this is the argument <i>of knowledge</i> .

Declaration	Module	Statement
ZkPoP.Schnorr.accepting_iff_sim	Schnorr	Every accepting transcript equals the witness-free simulator's output on its own responses: perfect honest-verifier zero knowledge.
ZkPoP.Shard.decode_encode	Shards	Lagrange decoding from any N distinct evaluation points recovers a message of degree $< N$: any N of the M carriers suffice.
ZkPoP.Shard.shamir_privacy	Shards	Any $N - 1$ shares are consistent with exactly one message for <i>every</i> candidate secret: sub-threshold carriers reveal nothing.
ZkPoP.ArgSystem.tower_sound	Recursion	Recursive self-verification of arbitrary depth preserves knowledge soundness.
ZkPoP.zk_acceptance_alone_insufficient	Recursion	There is a knowledge-sound, self-verifying system whose acceptance implies nothing: the proof layer transports soundness, it does not create it.
ZkPoP.Skill.end_to_end	Skill	For any argument system knowledge-sound for the circuit relation, an accepted proof implies the declaration's semantic claim.
ZkPoP.Skill.end_to_end_recursive	Skill	The same conclusion survives recursion to arbitrary depth (self-hosting).
ZkPoP.Skill.end_to_end_library	Skill	One aggregated argument implies the semantic claim of every declaration in the library.
ZkPoP.Skill.artifact_hides_witness	Skill	The published pair (commitment, transcript) is a function of public data alone.

4 Axiom closure

```

'ZkPoP.Tm.checker_sound' depends on axioms: [propext]
'ZkPoP.typeSat_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.typeSat_sound' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.rootR1CS_sat_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.horner_root_iff_trace' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.pedersen_perfectly_hiding' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.pedersen_binding_extract_dlog' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.pedersen_binding' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Schnorr.completeness' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Schnorr.special_soundness' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Schnorr.accepting_iff_sim' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Schnorr.simulator_verifies' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Shard.decode_encode' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Shard.decode_unique' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Shard.shamir_privacy' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.ArgSystem.tower_sound' does not depend on any axioms
'ZkPoP.ArgSystem.tower_complete' does not depend on any axioms
'ZkPoP.zk_acceptance_alone_insufficient' does not depend on any axioms
'ZkPoP.Skill.end_to_end' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Skill.end_to_end_recursive' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Skill.end_to_end_library' depends on axioms: [propext, Classical.choice, Quot.sound]
'ZkPoP.Skill.artifact_hides_witness' depends on axioms: [propext, Classical.choice, Quot.sound]

```

5 Measured behaviour of the artifact

N -of- M recovery, subsets tested	35 (all $\binom{7}{4}$)
of which reconstructed the exact package	35
Carriers surviving lossless PNG re-encoding	7 / 7
Carriers surviving half-scale down/up sampling	7 / 7
Carriers surviving JPEG re-compression, $q = 95/75/50/30/20$	7/7/7/7/7
Honest argument accepted	yes
Tampered response rejected	yes
Wrong statement root rejected	yes
Wrong commitment rejected	yes
Wrong context rejected	yes
Extractor recovers the opening from two transcripts	yes

These are measurements of the reference implementation in `tools/`; the *theorems* they instantiate are the Lean results in the previous section.

6 Carrier construction

The package is compressed and split by a systematic Cauchy Reed–Solomon code over $\text{GF}(256)$: the generator matrix is $[I \ ; \ C]$ with C Cauchy, so every selection of N rows is invertible and any N carriers decode. Each shard is framed with a magic string, its index, the code parameters, the payload length and a CRC-32, then drawn as a grid of 10×10 pixel cells with a quiet margin. A reader averages the central part of each cell and thresholds, so the payload survives re-compression; the CRC lets a damaged carrier be discarded and the erasure code covers the loss.

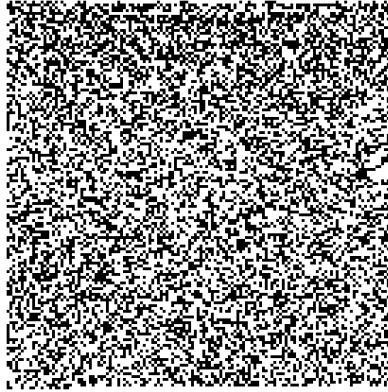


Figure 1: One of the 7 carriers. Each cell is a bit; any 4 carriers reconstruct the package, any 4–1 of them reveal nothing.

The two mathematical guarantees are Lean theorems: `ZkPoP.Shard.decode_encode` (any N carriers reconstruct the payload exactly) and `ZkPoP.Shard.shamir_privacy` (any $N - 1$ carriers are consistent with every candidate payload, hence reveal nothing).

7 Trust boundary

The following remain outside what is proved here and must be trusted, audited or separately formalised:

- the hardness assumption (discrete logarithm in the chosen group) behind computational binding, and the random-oracle heuristic behind Fiat–Shamir;

- the correspondence between the object language of the Lean development and any larger system one wishes to certify: scaling the semantic bridge to a full dependent-type kernel is not done here;
- the reference implementation in `tools/` – it instantiates the proved protocol, but the code itself is not verified;
- collision-freeness of SHA-256, modelled in Lean by injectivity of the digest in `ZkPoP.Skill.statement_binding`.

8 Reproducing

```
lake build RequestProject.Main      # rebuild and re-check every theorem
python3 tools/zk_skill.py build     # extract statements, commit, prove, shard
python3 tools/zk_skill.py verify    # verify the argument and the statement root
python3 tools/zk_skill.py recover 4 -- rebuild the package from N carriers
python3 tools/selftest.py           # all measurements in this report
python3 tools/make_paper.py         # regenerate this PDF from the manifest
```